PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Milestone #4

mini Database Management System Development

Reported By:
GROUP MahSQL.5
assembly

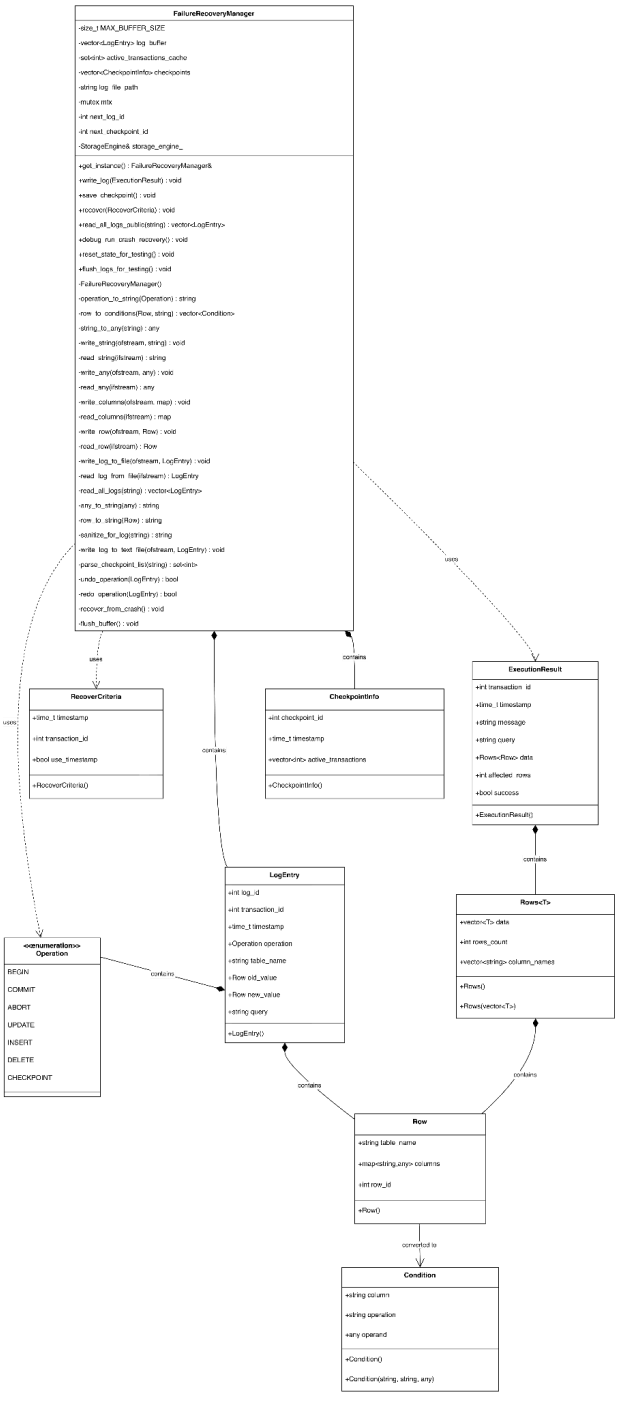
Anggota:
1. Alvin Christopher Santausa - 13523033
2. Muhammad Iqbal Haidar - 13523111
3. Ferdin Arsenarendra Purtadi - 13523117
4. Reza Ahmad Syarif - 13523119

IF3140 - Sistem Basis Data

2025

Part I. Gambaran Umum Komponen

Berikut *class diagram* Failure Recovery Manager



Gambar *Class Diagram* Failure Recovery Manager

Kelas	Kode	Fungsi
FailureRecoveryManager	class FailureRecoveryManager { public: static FailureRecoveryManager& get_instance(); FailureRecoveryManager(const FailureRecoveryManager&) = delete; FailureRecoveryManager& operator=(const FailureRecoveryManager&) = delete; ~FailureRecoveryManager(); void write_log(const ExecutionResult& info); void save_checkpoint(); void recover(const RecoverCriteria& criteria); void commit_transaction(int transaction_id); void abort_transaction(int transaction_id); void prepare_ddl_operation(const std::string& table_name, Operation op); private: FailureRecoveryManager(); const size_t MAX_BUFFER_SIZE = 50; std::vector<LogEntry> log_buffer; std::set<int> active_transactions_cache; std::vector<CheckpointInfo> checkpoints; std::string log_file_path; std::mutex mtx; int next_log_id; int next_checkpoint_id; sm::StorageEngine& storage_engine_; std::unordered_map<std::string, TableSchema> pending_drop_schemas_;	Kelas ini berfungsi sebagai pusat kendali proses pencatatan log, pembuatan checkpoint, dan pemulihan. Kelas FailureRecoveryManager memiliki beberapa method yaitu: 1. **get_instance()** : Mengembalikan single instance dari FailureRecoveryManager (Singleton pattern). 2. **write_log(info: ExecutionResult)** : Mencatat operasi database ke log buffer. Menentukan tipe operasi (INSERT/UPDATE/DELETE/BEGIN/COMMIT/ABORT), menyimpan old_value dan new_value, lalu menambahkan ke buffer. Memanggil flush jika buffer mencapai kapasitas maksimum. 3. **save_checkpoint()** : Membuat checkpoint dengan melakukan flush semua buffer ke disk, mencatat transaksi aktif, dan menyimpan checkpoint info untuk recovery point. 4. **recover(criteria: RecoverCriteria)** : Melakukan recovery dengan UNDO operasi secara backward berdasarkan transaction_id. Membaca semua log dari disk, filter berdasarkan kriteria, lalu rollback operasi dari yang terbaru ke awal. 5. **commit_transaction(transaction_id: integer)** : Fungsi ini memfinalisasi transaksi dengan menulis status COMMIT ke dalam log untuk memastikan semua perubahan data tersimpan secara permanen.

	`` std::thread checkpoint_thread_; std::atomic<bool> running_; const int CHECKPOINT_INTERVAL_SECONDS = 300; void checkpoint_worker(); operation_to_string(Operation op); std::vector<Condition> row_to_conditions(const Row& row, const std::string& table_name); std::any string_to_any(const std::string& str); void write_string(std::ofstream& out, const std::string& str); std::string read_string(std::ifstream& in); void write_any(std::ofstream& out, const std::any& val); std::any read_any(std::ifstream& in); void write_columns(std::ofstream& out, const std::map<std::string, std::any>& columns); std::map<std::string, std::any> read_columns(std::ifstream& in); void write_row(std::ofstream& out, const Row& row); Row read_row(std::ifstream& in); void write_schema(std::ofstream& out, const TableSchema& schema); TableSchema read_schema(std::ifstream& in); LogEntry read_log_from_file(std::ifstream& in); ```	6. **abort_transaction(transaction_id: integer)** : Fungsi ini membatalkan transaksi yang sedang berjalan dengan menulis status ABORT ke log dan mengembalikan data ke kondisi sebelum transaksi dimulai (rollback) 7. **prepare_ddl_operation(table_name: string, op: Operation)** : Fungsi ini menyiapkan dan mencatat operasi perubahan struktur database (seperti membuat atau menghapus tabel) ke dalam log untuk menjamin konsistensi skema saat pemulihan. 8. **checkpoint_worker()** : Fungsi ini berjalan secara otomatis di latar belakang (background thread) untuk melakukan proses checkpoint secara berkala guna menyinkronkan data dari memori ke disk. 9. **operation_to_string(op: Operation)** : Mengkonversi enum Operation menjadi string (BEGIN/COMMIT/ABORT/INSERT/ UPDATE/DELETE/CHECKPOINT). 10. **row_to_conditions(const Row& row, const std::string& table_name)** : Mencari identifying condition (primary key) dari sebuah row pada tabel 11. **string_to_any(const std::string& str)** : Mengkonversi string menjadi std::any tipe data yang diinferensi. 12. **write_string(std::ofstream& out, const std::string& str)** : Menulis data string dalam bentuk binary ke file.

	`` void write_log_to_file(std::ofstream& out, const LogEntry& entry); std::vector<LogEntry> read_all_logs(const std::string& file_path); std::string any_to_string(const std::any& val); std::string row_to_string(const Row& row); std::string schema_to_string(const TableSchema& schema); std::string sanitize_for_log(std::string input); void write_log_to_text_file(std::ofstream& out, const LogEntry& entry); std::set<int> parse_checkpoint_list(const std::string& query) bool undo_operation(const LogEntry& entry); bool redo_operation(const LogEntry& entry); void recover_from_crash(); void flush_buffer(); ```	13. **read_string(std::ifstream& in)** : Membaca data string dalam bentuk binary dari file. 14. **write_columns(std::ofstream& out, const std::map<std::string, std::any>& columns)** : Menulis data map columns dalam bentuk binary ke file. 15. **read_columns(std::ifstream& in)** : Membaca data map columns dalam bentuk binary dari file. 16. **write_row(std::ofstream& out, const Row& row)** : Menulis data Row dalam bentuk binary ke file. 17. **read_row(std::ifstream& in)** : Membaca data Row dalam bentuk binary dari file. 18. **write_schema(std::ofstream& out, const TableSchema& schema)** : Menulis data TableSchema dalam bentuk binary ke file. 19. **TableSchema read_schema(std::ifstream& in)** : Membaca data TableSchema dalam bentuk binary dari file. 20. **read_log_from_file(std::ifstream& in)** : Membaca sebuah data LogEntry dalam bentuk binary dari file. 21. **write_log_to_file(std::ofstream& out, const LogEntry& entry)** : Menulis sebuah data LogEntry dalam bentuk binary ke file. 22. **read_all_logs(const std::string& file_path)** : Membaca seluruh data LogEntry dalam bentuk binary dari file.

		23. **any_to_string(const std::any& val)** : Mengkonversi `std::any` menjadi string untuk serialisasi (mendukung tipe int, float, dan string). 24. **row_to_string(const Row& row)** : Melakukan serialisasi Row menjadi format string. 25. **schema_to_string(const TableSchema& schema)** : Melakukan serialisasi TableSchema menjadi format string. 26. **sanitize_for_log(std::string input)** : Fungsi helper untuk mengganti newline menjadi spasi pada string. 27. **write_log_to_text_file(std::ofstream& out, const LogEntry& entry)** : Menulis sebuah data LogEntry dalam bentuk plaintext ke file. 28. **flush_buffer()** : Menulis semua log entries di buffer ke disk file, lalu mengosongkan buffer untuk menghemat memory. 29. **undo_operation(entry: LogEntry)** : Melakukan UNDO operasi berdasarkan tipe: - **INSERT** → DELETE row baru (new_value) - **DELETE** → INSERT kembali row lama (old_value) - **UPDATE** → Restore ke nilai lama (old_value) 30. **redo_operation(entry: LogEntry) :** Melakukan REDO operasi berdasarkan tipe: - **INSERT** → INSERT row baru (new_value)

		- **DELETE** → Hapus kembali row lama (old_value) - **UPDATE** → Update ke nilai baru (new_value) 31. **recover_from_crash() :** Melakukan recovery dari kegagalan sistem. Dimulai dengan proses mencari checkpoint terakhir, build undo_list. Kemudian, scan log setelah checkpoint untuk update status transaksi. Lalu, step REDO dan UNDO transaksi. 32. **parse_checkpoint_list(const std::string& query) :** Melakukan pembentukan undo_list dari log CHECKPOINT.
LogEntry	`struct LogEntry {` `  int log_id;` `  int transaction_id;` `  std::time_t timestamp;` `  Operation operation;` `  std::string table_name;` `  Row old_value;` `  Row new_value;` `  std::string query;` `  std::optional<TableSchema> dropped_schema;` `  std::optional<TableSchema> created_schema;` `  LogEntry() : log_id(-1), transaction_id(-1), timestamp(std::time(nullptr)) {}` `};`	Kelas ini merepresentasikan satu entri log yang disimpan saat suatu transaksi melakukan operasi.
CheckpointInfo	`struct CheckpointInfo {` `  int checkpoint_id;` `  std::time_t timestamp;` `  std::vector<int> active_transactions;` `  CheckpointInfo() : checkpoint_id(-1), timestamp(std::time(nullptr)) {}`	Merepresentasikan checkpoint yang disimpan untuk mempercepat proses recovery.

	}; 	
RecoveryCriteria	struct RecoverCriteria { std::time_t timestamp; int transaction_id; bool use_timestamp; RecoverCriteria() : timestamp(0), transaction_id(-1), use_timestamp(false) {} }; 	Kelas ini digunakan untuk menentukan bagaimana proses recovery dilakukan.
Operation	enum class Operation { BEGIN, COMMIT, ABORT, UPDATE, INSERT, DELETE, CHECKPOINT, CREATE_TABLE, DROP_TABLE }; 	Enumerasi untuk mendefinisikan jenis operasi yang dicatat dalam log
Row	struct Row { std::string table_name; std::map<std::string, std::any> columns; int row_id; Row() : row_id(-1) {} }; 	Untuk merepresentasikan satu baris data dalam tabel

Part II. Bagian yang Telah Selesai

Berikut adalah bagian yang berhasil dikembangkan

- Inisialisasi tipe dan struktur data yang diperlukan untuk Failure Recovery
- Pembuatan Class Diagram
- Konstruksi method write_log
- Konstruksi method save_checkpoint
- Konstruksi method recover
- Konstruksi fitur Penyimpanan Log
- Konstruksi fitur Recovery

- Implementasi System Failure Recovery dengan redo-undo

Part III. Bagian yang Belum Selesai

Berikut tugas-tugas yang masih perlu diselesaikan :

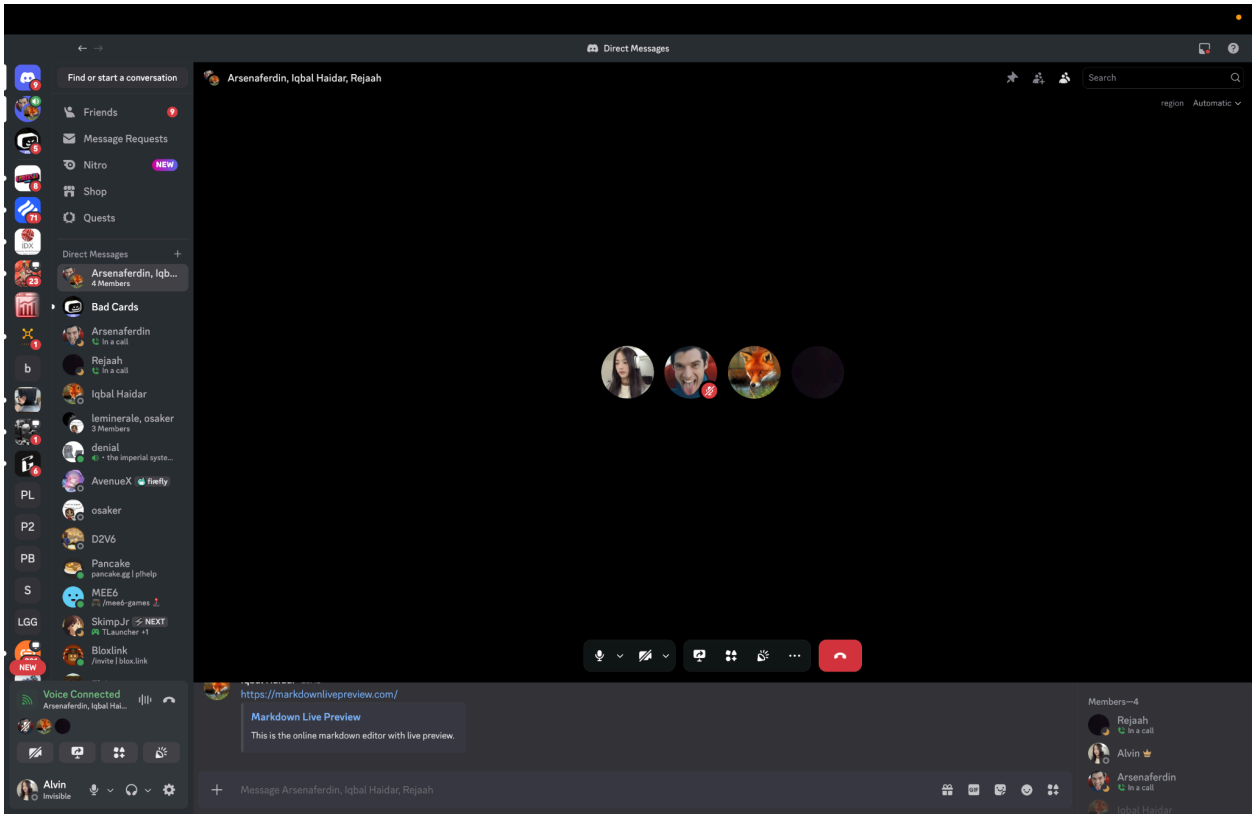
Part IV. Distribusi Workload

NIM	Nama	Workload
13523033	Alvin Christopher Santausa	Pembuatan laporan, integrasi, debugging, memastikan semua berjalan dengan benar.
13523111	Muhammad Iqbal Haidar	Pembuatan laporan, implementasi fitur penulisan/pembacaan TableSchema dalam bentuk binary ke/dari file.
13523117	Ferdin Arsenarendra Purtadi	Pembuatan laporan, implementasi fungsi write_log, menerjemahkan ExecutionResult dari Query Processor menjadi logEntry.
13523119	Reza Ahmad Syarif	Pembuatan laporan, implementasi fitur create and drop table recovery

Part V. Lampiran

Sertakan lampiran yang diperlukan. Lampiran wajib adalah **tautan *GitHub Release*** untuk setiap *checkpoint*.

Github Release:


Direct Messages
Find or start a conversation
Friends
Message Requests
Nitro
NEW
Shop
Quests
Direct Messages
Arsenaferdin, Iqb...
4 Members
Bad Cards
Arsenaferdin
In a call
Rejaah
In a call
Iqbal Haidar
leminerale, osaker
3 Members
denial
AvenueX
firefly
osaker
D2V6
Pancake
pancake.gg | pfhelp
MEE6
SkimpJr
NEXT
Bloxlink
/invite | bloxlink
Arsenaferdin, Iqbal Haidar, Rejaah
Search
region
Automatic
https://markdownlivepreview.com/
Markdown Live Preview
This is the online markdown editor with live preview.
Message Arsenaferdin, Iqbal Haidar, Rejaah
Members—4
Rejaah
In a call
Alvin
Arsenaferdin
In a call
Voice Connected
Arsenaferdin, Iqbal Hai...
Alvin
Invisible